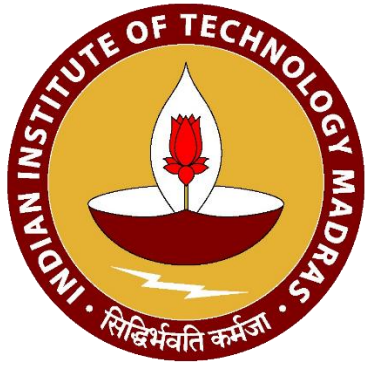
Python for Data Science
Web M. Tech Course DA5301W
Module 12: Tensorflow and PyTorch

Dr. P.S Jayadev



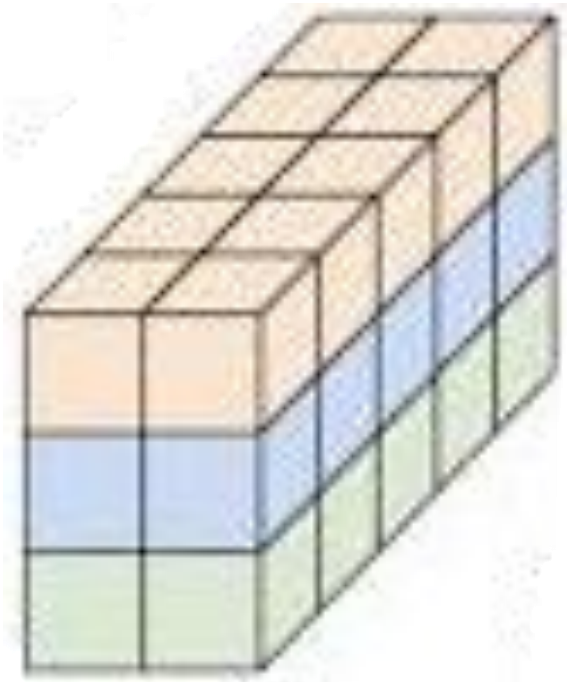
Contents

- What is a Tensor?
- Computational Graphs on Tensors
- Pytorch and its Features
- Tensorflow and its Features

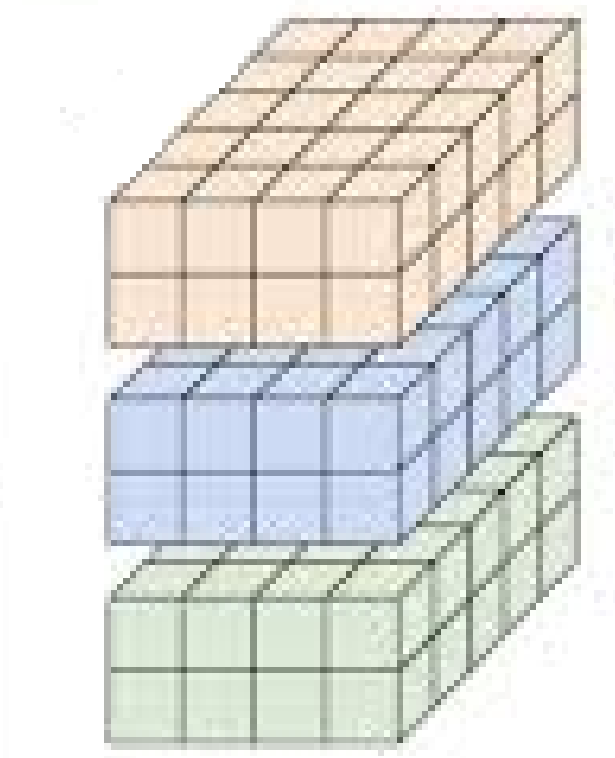
Tensor

- Tensor is a multi-dimensional array that generalizes many other data forms
- Generalizes scalars (0D), vectors (1D), matrices (2D), and higher-dimensional arrays (3D and above)
- Tensors can hold elements of a homogeneous data type, such as integers, floats or complex numbers
- Rank of a Tensor: No. of dimensions (axes) a tensor has
- Shape of a tensor: Size of the tensor along each axis (same as in numpy)
- Examples:
  - Scalar: A single number, rank 0: 7
  - Vector: A one-dimensional array, rank 1 and shape (3,): [1, 2, 3]
  - Matrix: A 2-dimensional array, rank 2 and shape (2, 2): [[1, 2], [3, 4]]
  - 3D Tensor: A 3-dimensional array, rank 3 and shape (2, 2, 2): [[[1, 2], [3, 4]], [[5, 6], [7, 8]]]
- Math Operations: support a variety of math operations, including addition, multiplication, and supports hardware acceleration through frameworks like tensorflow/pytorch

Tensors Visualization



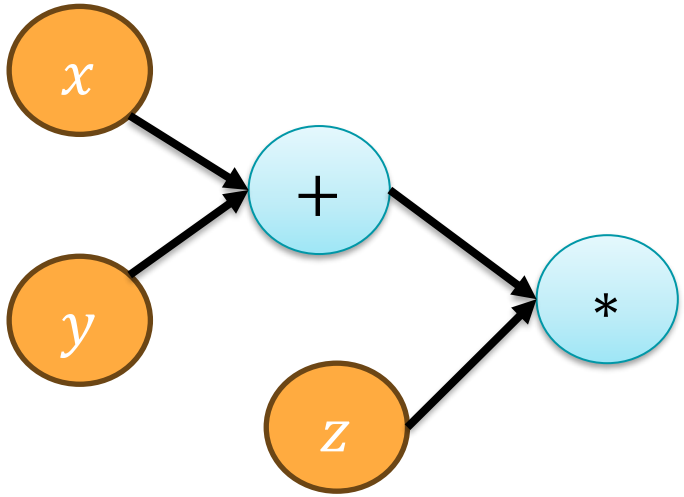
Rank-3 tensor of shape [3,2,5]



Rank-4 tensor of shape [3,2,4,5]

Computational Graph

- Directed graph (DAG) which is useful in performing computations in a memory and time efficient manner
- Nodes: Represent data (tensors) or functions/operations (e.g., addition, multiplication, neural network layers)
- Edges: Represent data dependency or data flowing between operations
- Example: $f(x, y, z) = (x + y) * z$



Types of Computational Graphs

Static Computation Graph

- Also known as declarative graphs
- Entire graph structure is defined before any computation starts
- Allows for optimization and efficient deployment as graph is known
- Debugging is hard because of static nature
- Less flexible
- Tensorflow 1.x uses this type of graph

Dynamic Computation Graph

- Also known imperative graphs
- Graph structure is defined dynamically during execution
- Less optimized performance compared to static graphs
- Debugging is easy as computations can be inspected during execution
- More flexible
- Pytorch and Tensorflow 2.0 use this type of graph

Pytorch

Features of PyTorch

- Easy to Learn for python programmers: Syntax and APIs are intuitive and similar to Python's standard libraries
- GPU Support: PyTorch seamlessly integrates with CUDA, allowing for efficient computation on GPUs
- Rich Ecosystem: Extensive libraries and tools (like torchvision for vision tasks) are available to extend PyTorch’s functionality

Popular Libraries in Pytorch Ecosystem

- TorchVision: For computer vision tasks
- TorchText: Specialised for NLP tasks
- TorchAudio: Specialised for audio and speech processing tasks
- Captum: For model interpretability and explainability
- Detectron: For Object Detection and Segmentation

Features of PyTorch

- Easy to Learn for python programmers: Syntax and APIs are intuitive and similar to Python's standard libraries
- GPU Support: PyTorch seamlessly integrates with CUDA, allowing for efficient computation on GPUs
- Rich Ecosystem: Extensive libraries and tools (like torchvision for vision tasks) are available to extend PyTorch’s functionality
- Autograd: An automatic differentiation library in pytorch which automates the computation of gradients

Autograd

- Gradient: Indicates the impact of one variable on another i.e., how much is the change in variable y when x changes $\frac{\partial y}{\partial x}$
  - Gradient computations are inevitable in training of neural network models
  - Gradient guides in determining the model parameters for given input-output data
- Autograd:
  - Automates the process of gradient computation which makes it very convenient to train DL models
  - Computation of gradients happens based on computation graph

```python
import torch

# Create tensors
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
y = x * 2
z = y.sum()

# Perform backward pass
# This step computes gradient of z w.r.t x
z.backward()

# Print gradients
print(x.grad)  # Output: tensor([2., 2., 2.])
```

Features of PyTorch

- Easy to Learn for python programmers: Syntax and APIs are intuitive and similar to Python's standard libraries
- GPU Support: PyTorch seamlessly integrates with CUDA, allowing for efficient computation on GPUs
- Rich Ecosystem: Extensive libraries and tools (like torchvision for vision tasks) are available to extend PyTorch’s functionality
- Autograd: An automatic differentiation library in pytorch which automates the computation of gradients
- Optimizers: Provides standard optimizers used in training neural networks such as Adam, SGD, etc.

Challenges/Limitations of PyTorch

- Training: Managing training loops in pytorch can be tedious as each of the main steps needs to defined separately
- Logging: No build-in support for logging to monitor the training process and analyse the model
- Training on multiple GPUs:
  - Distributed model training on multiple GPUs is complex to implement
  - Debugging in a distributed setting is challenging
- Training models on TPUs:
  - Tensor Processing units specialized hardware for accelerating tensor operations involving matrices or higher dimensional arrays
  - TPUs are faster than GPUs when dealing with tensors especially for deep learning
  - Training models on TPUs is not straightforward using pytorch
- Pytorch Lightning to the rescue!

Pytorch Lightning

- Pytorch Lightning is a wrapper around pytorch that makes the whole training process more streamlined and efficient
- Features of Pytorch Lightning:
  - Training: Provides a high-level API for model training which makes the code clearer and more concise
  - Logging: Integrates with popular logging frameworks to easily log training metrics
  - Training on multiple GPUs:
    - Simplifies distributed training by providing a unified interface
    - Provides tools and utilities to facilitate debugging in a distributed setting
  - Training models on TPUs: Supports training of models on TPUs with a simple implementation approach

Tensorflow

Features of Tensorflow

- High Performance: Optimized for running on multiple CPUs, GPUs, and even TPUs (Tensor Processing Units).
- Flexibility: Supports a wide range of APIs including high-level Keras API for easy model building to low-level operations for customization.
- Keras API: Equivalent to pytorch lightning in case of pytorch
- Automatic differentiation: Provides a simple API to compute gradients similar to autograd in pytorch

Automatic Differentiation

- Automates the process of gradient computation which makes it very convenient to train neural networks

```python
import tensorflow as tf

# Define a simple function
def f(x):
    return x**2 + 2*x + 1

# Define a variable
x = tf.Variable(3.0)

# Record the computation with GradientTape
with tf.GradientTape() as tape:
    y = f(x)

# Compute the gradient of y with respect to x
dy_dx = tape.gradient(y, x)

print(f"The gradient of y with respect to x is: {dy_dx.numpy()}")
```

GradientTape records all operations involving watched tensors and builds a DAG during execution

Features of Tensorflow

- High Performance: Optimized for running on multiple CPUs, GPUs, and even TPUs (Tensor Processing Units).
- Flexibility: Supports a wide range of APIs including high-level Keras API for easy model building to low-level operations for customization.
- Keras API: Equivalent to pytorch lightning in case of pytorch
- Automatic differentiation: Provides a simple API to compute gradients similar to autograd in pytorch
- Extensive Ecosystem: Includes tools like TensorBoard for visualization, TensorFlow Lite for mobile and embedded devices, and TensorFlow Extended (TFX) for production ML pipelines

TensorBoard

- Visualization tool to facilitate the understanding, debugging, and optimization of models developed using TensorFlow

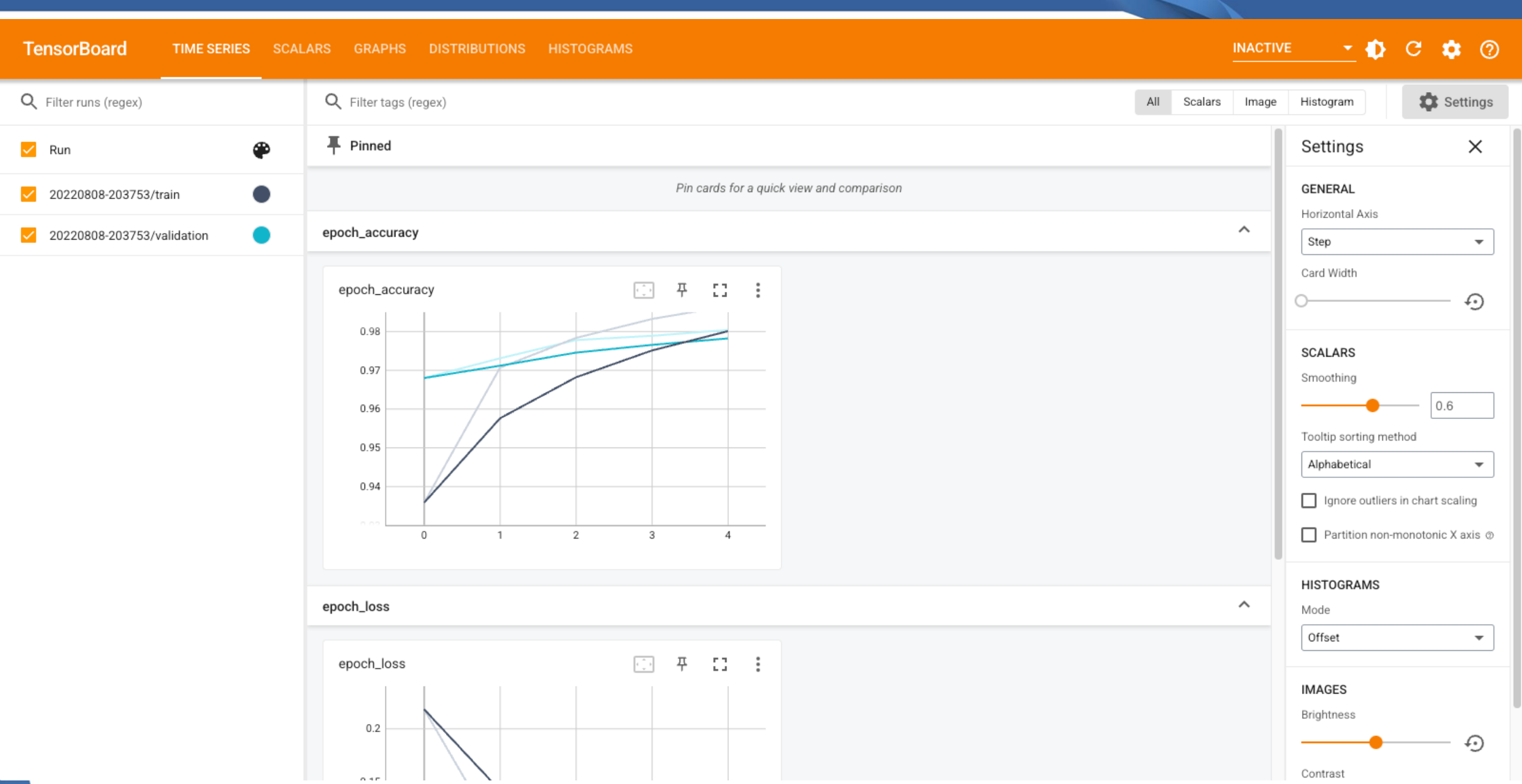


Image Source: tensorflow.org

TensorBoard

- Visualization tool to facilitate the understanding, debugging, and optimization of models developed using TensorFlow
- Key Features:
  - Track metrics such as loss and accuracy over time, during training

```
Epoch 1/5
938/938 [==============================] - 2s 2ms/step - loss: 0.6955 - accuracy: 0.7618
Epoch 2/5
938/938 [==============================] - 2s 2ms/step - loss: 0.4877 - accuracy: 0.8296
Epoch 3/5
938/938 [==============================] - 2s 2ms/step - loss: 0.4458 - accuracy: 0.8414
Epoch 4/5
938/938 [==============================] - 2s 2ms/step - loss: 0.4246 - accuracy: 0.8476
Epoch 5/5
938/938 [==============================] - 2s 2ms/step - loss: 0.4117 - accuracy: 0.8508
<tensorflow.python.keras.callbacks.History at 0x7f656ecc3fd0>
```

Image Source: tensorflow.org

TensorBoard

- Visualization tool to facilitate the understanding, debugging, and optimization of models developed using TensorFlow
- Key Features:
  - Track metrics such as loss and accuracy over time, during training
  - Visualise the computation graph of the model

Computation Graph on TensorBoard

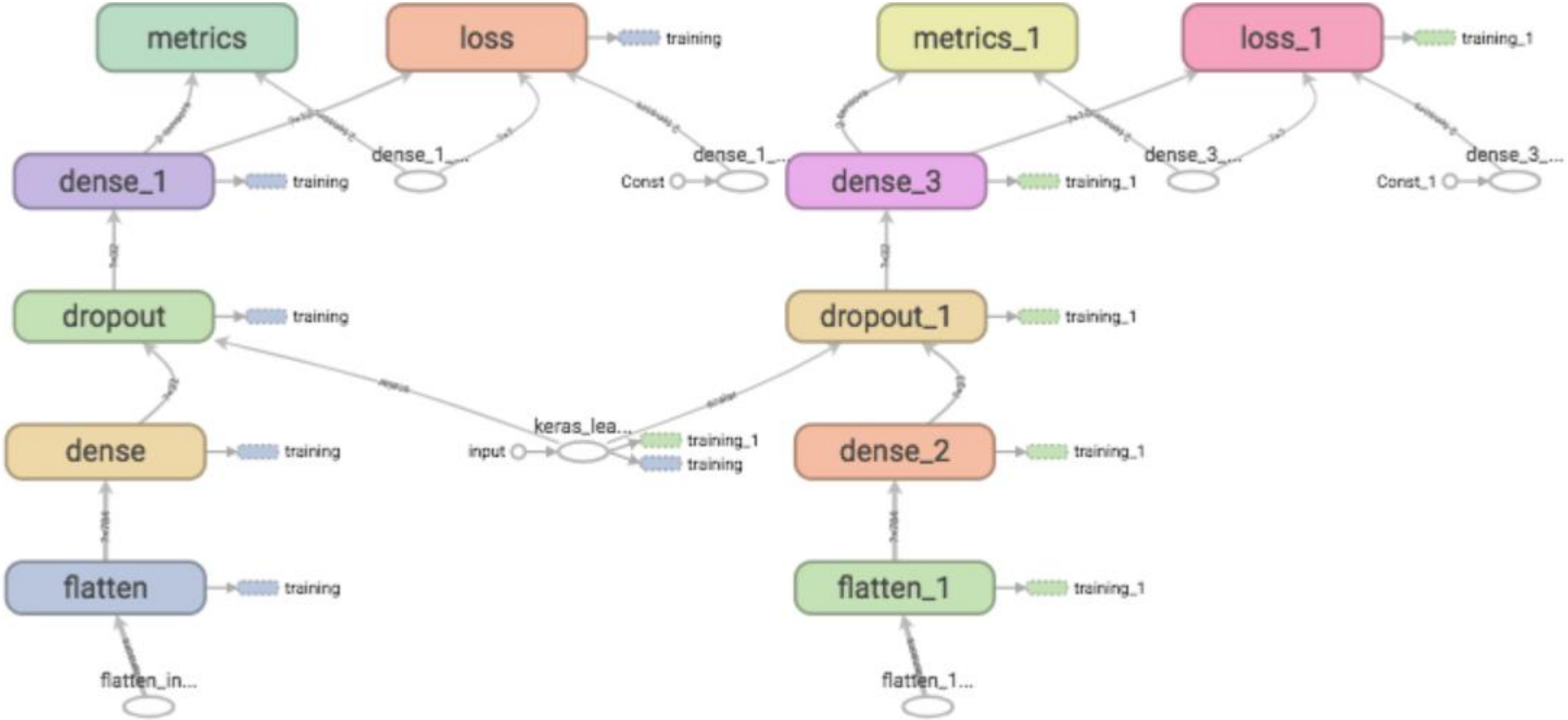


Image Source: tensorflow.org

TensorBoard

- Visualization tool to facilitate the understanding, debugging, and optimization of models developed using TensorFlow
- Key Features:
  - Track metrics such as loss and accuracy over time, during training
  - Visualise the computation graph of the model
  - Visualise the distribution of tensor values (weights) over time
  - For image datasets, visualise images passed during training and inference

TensorBoard IMAGES TIME SERIES

Filter runs (regex)

Run

20220809-140421

Filter tags (regex) All Scalars Image Histogram Settings

Pinned

Pin cards for a quick view and comparison

Training data

Training data

20220809-140421 Step 0

Settings

GENERAL

Horizontal Axis

Step

Card Width

SCALARS

Smoothing 0.6

Tooltip sorting method

Alphabetical

Ignore outliers in chart scaling

Partition non-monotonic X axis

HISTOGRAMS

Image Source: tensorflow.org

TensorBoard

- Visualization tool to facilitate the understanding, debugging, and optimization of models developed using TensorFlow
- Key Features:
  - Track metrics such as loss and accuracy over time, during training
  - Visualise the computation graph of the model
  - Visualise the distribution of tensor values (weights) over time
  - For image datasets, visualise images passed during training and inference
  - Tracking and visualize different runs of training with different values of hyperparameters

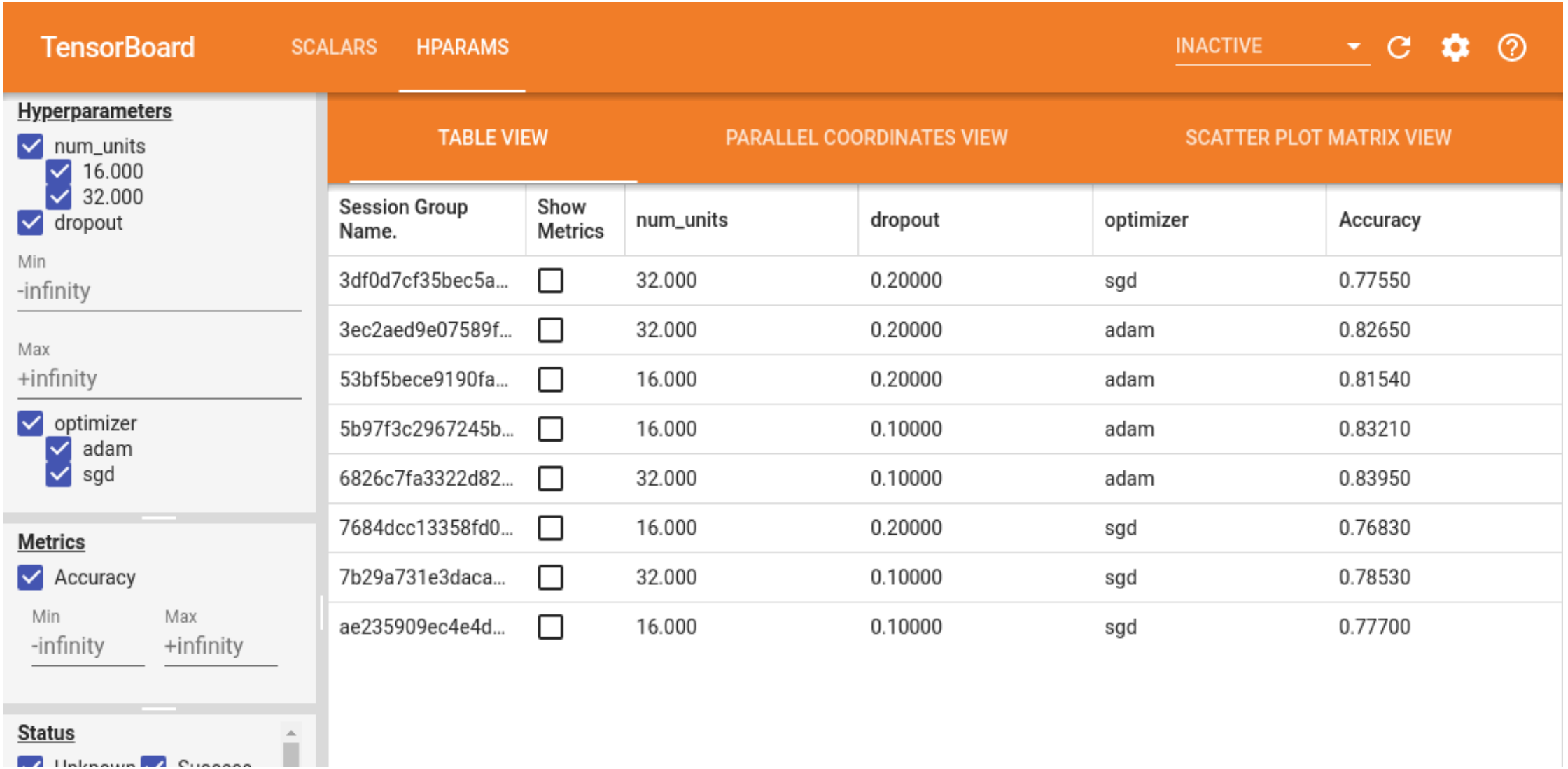


Image Source: tensorflow.org

TensorBoard

- Visualization tool to facilitate the understanding, debugging, and optimization of models developed using TensorFlow
- Key Features:
  - Track metrics such as loss and accuracy over time, during training
  - Visualise the computation graph of the model
  - Visualise the distribution of tensor values (weights) over time
  - For image datasets, visualise images passed during training and inference
  - Tracking and visualize different runs of training with different values of hyperparameters
  - Analyse performance bottlenecks in model training process (memory and CPU usage statistics)

TensorBoard SCALARS GRAPHS DISTRIBUTIONS HISTOGRAMS PROFILE INACTIVE

CAPTURE PROFILE

Runs (2)

20200314-180936/train/2020_03_...

Tools (5)

overview_page

Hosts (1)

72cefc0dc85c

Performance Summary

Average Step Time 30.1 ms
lower is better
(σ = 4.6 ms)

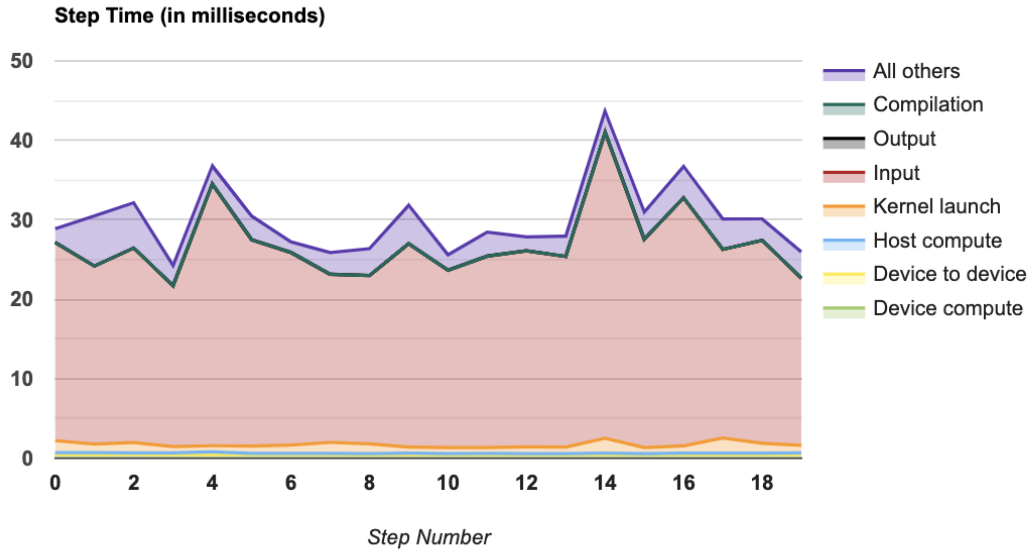
- **All Others Time** 3.2 ms
  (σ = 1.3 ms)
- **Compilation Time** 0.0 ms
  (σ = 0.0 ms)
- **Output Time** 0.0 ms
  (σ = 0.0 ms)
- **Input Time** 25.2 ms
  (σ = 4.4 ms)
- **Kernel Launch Time** 1.1 ms
  (σ = 0.4 ms)
- **Host Compute Time** 0.2 ms
  (σ = 0.1 ms)
- **Device to Device Time** 0.0 ms
  (σ = 0.0 ms)
- **Device Compute Time** 0.4 ms
  (σ = 0.0 ms)

Device Compute Precisions
(out of Total Device Time)
- 16-bit: 0.0
- 32-bit: 100.0

Run Environment

Number of Hosts used: 1

Step-time Graph



Recommendation for Next Step

- Your program is HIGHLY input-bound because 83.9% of the total step time sampled is waiting for input. Therefore, you should first focus on reducing the input time.
- 3.6 % of the total step time sampled is spent on Kernel Launch.
- 10.5 % of the total step time sampled is spent on All Others time.
- Only 0.0% of device computation is 16 bit. So you might want to replace more 32-bit Ops by 16-bit Ops to improve performance (if the reduced accuracy is acceptable).

Tool troubleshooting / FAQ
- Refer to the TF2 Profiler FAQ

Image Source: tensorflow.org

Tensorflow Lite

- Lightweight version of TensorFlow to enables machine learning inference with low latency on edge devices or resource constrained environments
- Key Features:
  - Model Conversion: Converts tensorflow models to tensorflow lite format (.tlite) to optimize the model size
  - Liteweight interpreter: Fast and efficient interpreter designed to run inference on devices with limited computational power and memory
  - Mobile Platforms: Supports android and IOS with dedicated APIs to integrate ML models into mobile applications
  - GPU and TPU support: Utilises GPU or TPU on the edge or mobile device if available
  - On-Device training: Supports on-device training and tuning of models on new data coming from the edge device
  - Pre-trained models: provides a wide range of pretrained models that are optimized for mobile and edge devices

Tensorflow Extended (TFX)

- End-to-end platform for deploying production machine learning (ML) pipelines
- Key Features:
  - Allows for seamless transition between data processing, model training, evaluation, deployment and monitoring
  - Consists of standard components responsible for specific task in the ML lifecycle
  - TFX integrates seamlessly with TensorFlow Serving, a flexible serving system for deploying ML models in production

Features of Tensorflow

- High Performance: Optimized for running on multiple CPUs, GPUs, and even TPUs (Tensor Processing Units).
- Flexibility: Supports a wide range of APIs including high-level Keras API for easy model building to low-level operations for customization.
- Keras API: Equivalent to pytorch lightning in case of pytorch
- Automatic differentiation: Provides a simple API to compute gradients similar to autograd in pytorch
- Extensive Ecosystem: Includes tools like TensorBoard for visualization, TensorFlow Lite for mobile and embedded devices, and TensorFlow Extended (TFX) for production ML pipelines
- Tensorflow Hub: Repository of ML modules developed using tensorflow and which can be re-used for similar tasks

Features of Tensorflow Hub

- Similar to transfer learning but a pipeline itself can be modularized and saved in tensorflow hub
- Hosts a variety of pre-trained modules, which can be used for tasks like image classification, object detection, text embedding, etc.
- Modules can be easily integrated into tensorflow and keras workflows
- Some modules also support fine tuning to adjust the weights of a pre-trained model
- Saves lot of time and computational resources